Biomedical Engineering

Final Year Project

Templating in Standard ML

Ja Ying Lew

May 2026

Supervisor: Gergely Buday

Dissertation submitted to the University of Sheffield in partial fulfilment
of the requirements for the degree of

(Bachelor of Engineering In Biomedical Engineering)

Abstract

Mustache is a logic-less templating language used widely in web development. Its multi-language design enforces a strict separation between templates and program logic. Template engines should not compute anything, only substitute values and iterate over data. This makes templates safe and easy to reason about, but it creates a problem. Any logic the template needs, whether an item is first in a list, whether a description is needed, or whether something is empty, has to be precomputed or specified after checking the output. As a result, this means someone has to manually add this in, which defeats the point of automation.

This dissertation aims to utilise standard ML's properties to preprocess the JSON before it is parsed through into the mustache template so that the final rendered output has 'logic'. The preprocessor sits in front of any standard Mustache implementation and requires no changes to the template layer.

The implementation uses CM-Lex and CM-Yacc to produce a abstract syntax tree (AST), with a lazy stream lexer feeding the parser. A renderer supports the core Mustache feature set, and the preprocessor handles null fields and array position flags. SML's type system provides compile-time verification that the required transformations are complete before rendering.

The prototype is tested with 47 unit tests across three suites covering the parser, preprocessor, and renderer, all passing on SML/NJ (Standard Meta Language/ New Jersey) version 110.99.9. Known limitations include parsed but unrendered support for partials and delimiter changes, hard-coded build paths, and a restricted variety of test inputs. The contribution is a working parser and preprocessor that demonstrates a potential in making templating safer and having 'logic'.

Acknowledgements

You may wish to include an acknowledgement. This should be a maximum of one page and is usually much less. Thank whoever may have helped you in any way, both serious and a bit of fun.

The acknowledgements are written in the first person singular, i.e. “I would like to thank my . . . ”

Contents

List of Figures

1. Introduction

Templating engines are one of the most common operations in modern software. Commonly used in web development, template engines are systems that generate outputs such as emails or documents from data files like JSON (JavaScript Object Notation). Any tool designed to generate source code from a specification will, at some point, separate a fixed structural skeleton from the variable data used to populate it. This separation is what a template system provides, enabling mass automation of tailored outputs. A practical example of this is a company that automates SDK (Software Development Kit) generation for its clients, since each client is different. Another example can be shown in 1.1, the left shows what the template would look like for the developer, and the right shows the generated output, which gets delivered to the customer, 'Sarah'.

Template

```
Dear {{name}},

Thank you for your order.
Your copy of {{book_title}}
will be delivered on
{{delivery_date}}.

Best wishes,
The Bookshop
```

Rendered email

Dear Sarah,

Thank you for your order. Your copy of Wuthering Heights will be delivered on 14 May.

Best wishes,
The Bookshop

Figure 1.1: A snippet template (left) and the email it produces when filled in with one customer's data (right). The double braces mark the slots where information is inserted from a dataset

Templating is embedded in our everyday lives: from web pages, emails, documents, and code generation. While famously used for web development, templating engines have expanded far beyond web development to DevOps and CI/CD pipelines [1], infrastructure as code [2], network automation [3], and general software tasks [4] and more.

Despite the significance of templating engines, there is a fair security issue that exists with them. Server-side template injections (SSTI) is a vulnerability which arises when user-supplied input is embedded into a template without being treated as protected data, allowing the attacker to inject executable code into the template engine [5]. While many developers confuse these flaws for cross-site scripting (XSS), which is an attack on the browser, exploiting vulnerabilities in client-side code, SSTI is more dangerous [6]. Being a risk highlighted by Open Worldwide Application Security Project (OWASP), SSTI is an attack on the server,

meaning the attacker may gain access to the server's file system and environment [7], [8]. The canonical study of the problem by Kettle [9] demonstrates that this typically can escalate to remote code execution (RCE), granting attackers full control over servers or applications. The real-world impact of SSTI is highlighted by the high-profile breach of uber in 2016, in which Orange Tsai exploited Flask's Jinja2 engine on 'rider.uber.com' by simply setting a profile name to a mathematical expression within a template tag, specifically ''7'*7', which the Uber mail server evaluated and returned as '7777777' in the confirmation email, earning him a $10,000 bug bounty for what was ultimately full remote code execution [10]. SSTI is not confined to one engine: documented exploits exist for Jinja2, Twig, Smarty, FreeMarker, Velocity, Thymeleaf, and Jade, among others, with many of the underlying engines downloaded millions of times a month [5], [11]. OWASP has listed SSTI vulnerabilities are among the most critical risks to the web applications for over a decade [7].

An overview of the security vulnerabilities of templating engines is discussed. The systematic analysis of 34 popular template engines across 8 programming languages found that 31/34 allowed RCE, while fewer than a third implement any built-in security protection, such as sandboxing or restricted execution [5]. Even where defences exist, they are routinely bypassed: a 2023 study of 7 widely-used PHP template engines identified 135 new template-escape bugs and synthesised working RCE exploits against 55 of them [12]. The vulnerability is also not a rarity in practice: it accounted for a measurable share of critical-risk findings in a commercial web application audit [5]. Remediation tends to be slow, extending the window during which affected applications remain exposed.

In response to these security issues is to take away the template's ability to execute code or involve business logic. Thus, enforcing a strict separation between presentation and application logic was argued for [13] on grounds of maintainability and security. A need for a template that cannot compute or be tricked into computing something the developer did not intend. Mustache released by Chris Wanstrath in 2009, introduces a 'logic-less' framework to separate business logic from presentation through reducing the available operations to variable substitution, section iteration and no conditionals or loops [14]. Because the syntax is so restricted, the same Mustache template can be rendered by an independent implementations in dozens of programming languages, including Ruby, JavaScript, Java, PHP, Haskell, and OCaml [15], [16]. Specifically, the price of this safety is what the literature calls the *preprocessing burden*. Anything that the template would have computed itself in a logic-friendly system, the developer must now compute in advance and embed in the data. If a list needs commas between items but not after the last one, the developer has to walk the list manually and attach an 'isLast' flag to each element. If a field might be missing, the developer has to add a companion 'has_field' flag to make the template's conditional render without missing it. None of this is hard, but all of it is tedious, error-prone, and untyped. Nothing checks that the developer remembered to add every flag, and a missing flag is silent

at compile time and only visible by inspecting the rendered output.

Logic-less templates are 'dumb' enough to evade these security issues; however, they have argued to have a trade-off. Logic-friendly templates are convenient but expose applications to security attacks, whereas logic-less templates are safe but divide business logic from presentation logic of the template. Critics [17], [18] have made a similar point of 'the logic has to go somewhere' [19]: instead of a logic-friendly template engine, the developer has to now precompute the 'logic' and embed the data into the template. The cost of preprocessing has two components, and the relationship between them matters more than either in isolation. The runtime cost is small. Research on type-based security in templating frameworks illustrates how dramatically architecture determines runtime cost: Samuel et al. [20] report that compile-time, type-based approaches to context-sensitive sanitisation incur overheads of 3-10%, against 78% to over 500% for purely runtime approaches measuring the same property. Preprocessing JSON to add contextual flags is a similarly bounded, compile-time-friendly operation. The cost that actually persists, however, is the developer's. Every new template, every change to the underlying data model, and every newly-added field potentially introduces a fresh set of flags that must be identified, computed, and embedded into the rendering context by hand. A list that gains a new sorting requirement needs a new flag; a field that becomes optional needs a companion 'has_field' flag added wherever the template references it; a section that becomes nested needs position flags propagated through the structure. Nothing in the toolchain checks that this work has been done: a missing flag produces a stray comma or a malformed sentence in the rendered output, with no error message and no compile-time warning. The preprocessing burden of logic-less templating is therefore not primarily a performance problem but a correctness and verification problem that recurs across the lifetime of the application, and it is this cost that the present work addresses.

This dissertation investigates whether Standard ML, a functional language, can automate that preprocessing while preserving Mustache's logic-less safety benefits. Functional programming properties, such as algebraic data types for tree-shaped data, exhaustive pattern matching, and robust static typing, are as effective for preprocessing automation as they are for traditional parsing and transformation tasks. The contribution of this project is a working prototype. It consists of a Mustache parser, a JSON preprocessor, and a renderer that consumes the enriched data. The preprocessor sits in front of any standard Mustache implementation, requires no changes to the Mustache specification, and uses Standard ML's pattern-matching exhaustiveness check to ensure that every transformation is applied consistently across nested data structures. All 47 unit tests across the three components pass, and the tool runs end-to-end from the command line.

Chapter 2 reviews the literature on template systems, examining alternative engines across platforms and investigating the parsing theory that underpins their implementation. Chapter 3 defines the preprocessor requirements, breaks the problem into manageable components,

and establishes the evaluation criteria. Chapter 4 justifies the major design decisions: language choice, architectural approach, and implementation strategy, against the alternatives considered. Chapter 5 describes the implementation, documents the technical challenges encountered, and presents the testing methodology and results. Chapter 6 evaluates the prototype against its original aims, compares the typed preprocessing approach to manual alternatives, and identifies directions for further work.

1.1 Functional Languages Usage in Biomedical Engineering

Functional languages have a long history in scientific computing, and BioCaml [21] is the most visible example of their adoption in biology: a comprehensive OCaml library for sequence analysis, file format parsing, and high-performance genomics. The case for typed functional languages in bioinformatics has been argued in detail by Berenger *et al.* [22], who identify the same properties this dissertation relies on. Strong static typing and Hindley-Milner inference allow complex hierarchical data structures to be represented precisely and checked for consistency at compile time, while algebraic data types and pattern matching let programs follow the shape of the data, and the compiler's exhaustiveness check catches missing cases before a long-running analysis crashes on input it was not prepared for. The preprocessor described in this dissertation uses an algebraic data type to represent the parsed Mustache template and exhaustive pattern matching to guarantee that every contextual flag a template requires has a corresponding case in the preprocessing function: the same engineering values, applied to a different domain.

2. Literature Review

2.1 Template theory and Systems Design

The Model-View-Controller principle, articulated by Krasner and Pope (1988) in Smalltalk-80 (an object-oriented programming language developed in the 1970s [23]), established the foundational separation of concerns that underpins modern template systems. This introduced a three-way factoring: models that encapsulate the logic of the application domain, views render graphical representations, and controllers that mediate user input, illustrated in 2.1 [24]. Now, in modern web architectures, template systems function as 'View', tasked solely with transforming model data into user-facing documents. Ideally, this should restrict modifications to presentation (views) to be strictly independent of application behaviour (models). In practise, this means separation should exist as a distinction between application logic (the model) and the presentation markup/ template code (the view), illustrated by 2.1; in between this lies the Command-Line Interface (CLI) as the controller. Template systems separate presentation markup from application code through 'separation of concerns': business 'model' logic (data processing, calculations) remains distinct from presentation 'view' logic (display structure). This separation improves maintainability, reduces security vulnerabilities, and enables parallel development by allowing designers to modify presentation independently from programmers updating business logic [13]. However, as template engines evolved, many abandoned this strict separation in favour of developer convenience, motivated by 'processing burden'. In a logic-less system, the view layer is incapable of evaluating state; it cannot determine if a collection is empty, calculate list indices or identify certain cases. Consequently, all these presentation-specific states must be explicitly computed in the 'model' data structure prior to rendering. To bypass this, engines have begun allowing conditional logic and data manipulation directly within the view layer [25]. Recent research identified 'mismatched data contexts' and 'syntax misuse' as the dominant root causes of bugs in modern templating applications, causing 19.42% and 35.67%of bugs, respectively [25]. A 'mismatched data context' occurs when raw, unprepared 'data context prepared by the host frequently mismatches the semantic expectations of the template placeholders" [25]. Furthermore, "syntax misuse' is highly prevalent because developers attempt to implement complex conditional logic using limited, uncompiled syntax of template engines; an example includes "Handlebars does not support nesting mustache expressions" [25]. Due to the deferred validation of template engines, these data mismatches and syntax bugs only reveal themselves as silent rendering errors at runtime. This architectural limitation directly motivates the selection of Standard ML for this project. By handling the preprocessing burden within SML, its strict static compiler can catch these issues as explicit compile-time warn-

ings long before the template is rendered, a structural advantage that will be detailed further in the system design section.

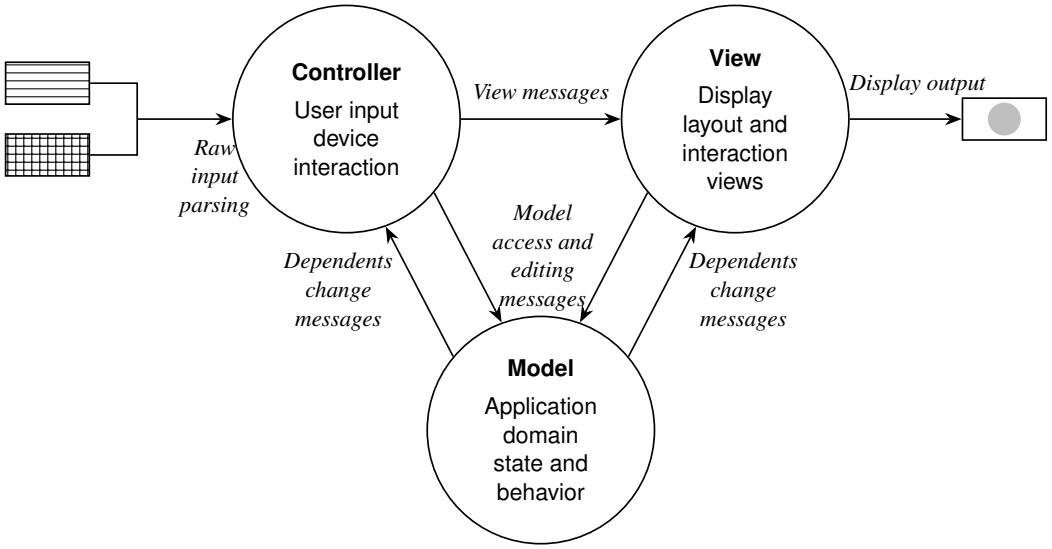

Figure 2.1: Model-View-Controller State and Message Sending (adapted from Krasner & Pope, 1988)

To understand why the industry drifted towards these vulnerabilities, understanding the preprocessing burden imposed by logic-less templates is key. While systems like Mustache successfully enable the separation of business logic within the view layer, they do not eliminate bugs but shift the point of failure to the host language. Because a logic-less view cannot evaluate state, all presentations must be explicitly predefined. Delegating these manual tasks to the model layer necessitates creating bloated view models, which may result in an inverse separation effect, where business logic is polluted by presentation constraints. This ends up being exhaustive and tedious, but leads to being highly error-prone. Because traditional architectures lack an enforceable interface to verify that the prepared data satisfies the template's semantic requirements, a vulnerability called 'Opaque Data Flow'; any human error during this manual preparation inevitably manifests as a 'Mismatched Data Context' rendering failure [25].

To escape the preprocessing burden, the industry drifted towards more computationally powerful templates (e.g. Jinja2), allowing developers to embed logic into the view. However, this drift fundamentally goes against the MVC architecture [13]. This review evaluates this structural failure through Parr's (2004) framework, which defines 5 "entanglement" violations conditional logic based on view structure, method/function calls from templates, value computation within templates, model scope manipulation, and recursive rendering [13]. Parr shows that unrestricted templates inevitably violate separation when developers embed business logic, as templates end up producing logic, with template computational power classified from weakest (substitution-only) to strongest (turing-complete).

Beyond rendering bugs, this entanglement introduces the severe security vulnerabilities high-

lighted previously. Pisu et al's (2025) evaluation of 34 template engines confirms that granting templates computational power directly links to SSTI to escalate to RCE [5]. Engines permitting complex logic and object access (e.g Jinja2, EJS) are highly susceptible to these fatal exploits, even suffering documented sandbox bypasses [5]. This highlights a fundamental dilemma: strict separation (Mustache) enforces security but imposes a severe, error-prone preprocessing burden. On the other hand, unrestricted power (Jinja2) reduces preprocessing at the cost of critical vulnerabilities and entanglement [13]. Therefore, an effective template system must satisfy three key design criteria [13]:

1. It must enforce the "separation of concerns", ensuring business logic remains in the model.
2. It must provide clear syntactic boundaries that distinguish presentation structure from application data.
3. It must balance computational power appropriately, restricting it to only what is necessary for presentation while avoiding unnecessary complexity in the model layer.

Having established the theoretical requirements for strict model-view separation and the security implications of entanglement, it is necessary to examine how current industry tools navigate this architectural dilemma.

2.2 Template Landscapes

The variation between logic-friendly and logic-less design recurs across every major programming platform. Specifically Parr identifies Entanglement Index ranging from 1 to 5 of which template engines are usually polarised t either Index 1 (strictly separated) or Index 5 (totally entangled). Thus, it can be identified that the problem is the tension of this balance [13].

2.2.1 Index 1: Logic-less systems

At Index 1, these are the engines that strictly forbid logic, math, or conditionals within the template and are the most strict on separation.

- **Mustache and Handlebars:** Mustache's widespread adoption across a dozen languages demonstrates a demand for a logic-less template engine, independent of any particular language's syntax [26]. Handlebars extends Mustache by adding helpers, block expressions, and partials, compiling templates into JavaScript functions for faster execution [27]. However, helper proliferation in Handlebars occurs when developers write dozens of custom helper functions to perform basic operations (comparisons, arithmetic, formatting) that the logic-less syntax prohibits, effectively rebuilding a programming language through helpers while claiming the templates remain "logic-less."

- **Liquid:** Developed by Shopify, it was designed to specifically to allow untrusted third-party authors to be able to customise e-commerce themes without compromising the server [28]. This is aimed to be secure, and each theme does not directly impact the others.

2.2.2 Index 2: Logic Friendly with Sandboxing

At Index 2, these engines allow view logic (like loops and conditionals) but attempt to contain it within a restricted environment to prevent system access and reduce preprocessing burden.

- **Twig and Nunjucks:** In a widely-used, open-source server-side scripting language designed primarily for web development, Hypertext Preprocessor (PHP), Twig was designed with an explicit sandbox that prohibits PHP execution to enforce the separation of concerns [29]. Similarly, Nunjacks (inspired by Python's Jinja2) provides block inheritance, autoscraping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining a readable syntax [30]. It offers template inheritance with block over- riding and automatic HTML escaping for XSS prevention.
- **Jinja2:** Jinja2 is Python's fast and extensible templating engine that supports template inheritance, macros, HTML auto escaping, and sandboxed environments for safely rendering untrusted templates [3], [18]. However, the sandbox approach is fragile and prone to bypassing security. For instance, CVE-2019-10906 demonstrated an incomplete blacklist; attackers bypassed the explicit block on 'str.format' by using 'str.format_map', traversing Python's object hierarchy to reach 'builtins' and achieve Remote Code Execution (RCE)[31]. More recently, CVE-2024-56326 highlighted the difficulty of static analysis in dynamic languages, where attackers passed an indirect reference to a malicious string's format method through a custom filter [32]. Also, Parr (2004) documents that "SquareSpace" was forced to abandon an unrestricted engine after attackers exploited templates to invoke malicious class loaders [13].

2.2.3 Index 5: Logic Friendly without Sandboxing

At Index 5 with unrestricted power, these frameworks works at zero security isolation and operate at the highest level of entanglement.

- **Blade and Embedded Ruby (ERB):** In the PHP system, Laravel's native blade engine allows developers to embed arbitrary PHP expressions in the view via php directive, making the logic fully entangled. ERB similarly allows unrestricted Ruby execution within templates [33]. These systems offer maximum flexibility, prioritising developer speed over architectural separation.
- **Embedded JavaScript (EJS):** EJS allows developers to write JavaScript straight into

templates using special '<% %>' delimiters [34]. It utilises functions for fast execution, but provides no security boundaries, meaning they have direct access to JavaScript runtime, making it completely unsuitable for scenarios involving untrusted templated sources.

2.2.4 Language-Integrated Template (Type-Safe Merging

At Index 5, ...

- **React/JavaScript XML (JSX- JavaScipt extension) and Svelte:** JSX integrates markup directly into JavaScript a code through components [35]. Whereas Svelte shifts template processing from the browser runtime to compile-time, generating highly optimised JavaSciript without framewrok overhead [36].

While some type-safe systems solve some data flow issues, they still struggle with preprocessing burden. In Scala/ Play, teams adopting logic-less methods found themselves creating an entirely new architectural layer, the Presenter, solely hold logic the template could no longer express [37]. This presents a problem of presentation logic does not disappear.

System	**Computational Power**	**Preprocessing Burden**	**Security Model**	**Key Limitation**
Mustache	Substitution only	High (all logic precomputed)	No sandbox needed	Bloated view models
Handlebars	Limited helpers	Medium (helpers reduce burden)	Helper whitelisting	Helper proliferation
Nunjucks	Context-sensitive	Low (templates handle logic)	Sandboxing	Separation violations
Jinja2	Turing-complete	Minimal (full Python access)	Sandbox (bypassable)	SSTI vulnerabilities
EJS	Turing-complete	Minimal (full JavaScript)	None	Direct code injection
React/JSX	Turing-complete	Minimal (native JS)	Type checking only	Not logic-less
Svelte	Compile-time	Low (compiler handles logic)	Static analysis	Requires build step

Table 2.1: Template System Comparison

Table 2.1 summarises an inverse relationship in the templating landscape: strict separation enforcement correlates with a high preprocessing burden (Mustache, Handlebars), while unrestricted computational power correlates with low preprocessing but documented security vulnerabilities (Jinja2, EJS). Overall, it can be evaluated a gap remains unsolved: no existing logic-less system provides a automated support for the preprocessing transformations it

requires. This project aims to address that. In the next chapter, I will explore functional languages and how they might be used for this.

2.3 Functional Languages for Template Processing

Functional programming languages (Standard ML, OCaml, and Haskell) bring distinct characteristics to template processing: strong static typing catching errors at compile-time, pattern matching for data structure manipulation, and algebraic datatypes (ADTs) for representing abstract syntax trees [38], [39].

2.3.1 Haskell: Type-checked Metaprogramming

Haskell's lazy evaluation and advanced type system have produced two distinctive templating approaches **haskell**. **Hamlet**, part of the Yesod web framework, uses Template Haskell metaprogramming to compile templates at build time, enabling type-checked variable substitution and ensuring that templates referencing undefined fields fail to compile rather than at runtime [40]. This shifts an entire category of template errors from production to compilation, demonstrating how sufficiently powerful type system can guarantee template accuracy.

On the other hand, **Blaze-html** represents HTML directly as composable Haskell functions rather than as a separate template language [41], [42]. Templates are not strings to be parsed but rather value constructed through variables. This eliminates the parser entirely and provides maximum type safety, but the cost of abandoning the template-as-text model that makes templates accessible to non-programmers: a trade-off that mirrors React/JSX's design choices in the JavaScript ecosystem.

2.3.2 OCaml: Industrial Pragmatism

OCaml prioritises industrial pragmatism with aggressive compiler optimisations and richer standard library via OPAM (source based package manager): combining a type system with strict evaluation and emphasis on performance [39], [43]. The Jingoo libary provides Jinja2-compatible templating for OCaml applications, allowing developers to use Jinja2 syntax within OCaml programs [44]. While this offers developers familiarity from migrating from Python, it inherits Jinja2's security model, including the documented sandbox bypass concerns discussed in the earlier sections, demonstrating that functional languages hosting alone does not resolve the underlying security trade-offs of logic heavy syntax.

2.3.3 Elm in Elm-mustache: Closest to prior art

Elm-mustache is particularly relevant to this project because it is a strictly functional language with an existing Mustache implementation that explores an approach to the preprocessing problem. The library 'elm-mustache', developed by Emma Bastas and actively main-

tained as of 2024 [45], offers both a browser-based Mustache parser and renderer, and a command-line tool that generates type-safe Elm rendering functions from .mustache template files in build time. In its default typesafe mode, the tool produces a rendering function with an explicit context record type derived from the template's variable and section names, which the Elm compiler checks at compile time: a developer who forgets to supply a required field receives a type error before the program runs. This is the preprocessing burden made visible at the type level, exactly the guarantee that Mustache's logic-less design cannot provide when the data arrives as untyped JSON.

Gap

However, the typesafe mode cannot handle Mustache's list iteration. The reason is a genuine structural ambiguity: given a section with variables 'foo' and 'bar' inside it, the tool cannot determine whether those variables should be repeated for each iteration or whether they are context-wide constants, and therefore cannot produce a single correct 'Context' type without additional information [45]. To support iteration, the library offers a "full" mode that generates a runtime Mustache renderer accepts untyped Json ('Json.Decode.Value'), which supports iteration but loses the compile-time type guarantees.

This identifies the precise gap the present project fills: the typesafe approach demonstrated by 'elm-mustache' is useful for boolean sections but cannot generalise to list iteration, whereas a preprocessor that walks at typed JSON structure and augments it with contextual flags ('isFirst', 'isLast', 'hasfield') before handing it to a renderer addresses both list iteration and compile-time completeness guarantees without requiring the template to be known at build time.

2.3.4 Standard ML

Standard Meta Language (SML or Standard ML) provides a different outlook due to its complete specification [46]. Unlike the more expansive or experimental feature sets of OCaml or Haskell, SML's pattern matching is subject to rigorous exhaustiveness checking defined by its mathematical semantics [46], [47]. This ensures that every possible shape of data structure is accounted for during language processing. Its selection will be discussed and evaluated in design.

2.3.5 Takeaway

These alternatives reveal a persistent tension on the security-usability spectrum. Mustache prioritises security through logic-less design but at the cost of extensive manual data preparation [17]. Conversely, systems like Jinja2 reduce the preprocessing burden but introduce security vulnerabilities [5]. While the complaints regarding preprocessing are well-documented, no existing work formalizes these patterns into a type-safe, automated pipeline.

The "iteration gap" identified in elm-mustache suggests that a more mathematically rigorous approach to data transformation is required. Addressing this, this project investigates whether the formal properties of Standard ML can automate these transformations, like specifically handling the ambiguity of list iteration as well as preserving the security advantages of logic-less templates.

2.4 Parsing Theory and Strategies

Parsing theory defines how a sequence of characters (a string) is transformed/"parsed" into a structural representation (a tree): this breakdown is understandable by a computer. In the context of this project, it is the 'raw parsed input' into the controller, which then reads the AST in 2.1.

When you provide a mustache template, which looks like "sectionname/section", the computer recognises it as ASCII characters of which the parsing theory breaks this down.

- Lexer: This scans the group of characters into "tokens" ie OPEN_SECTION, NAME, CLOSE_SECTION.
- Parser: This arranges those tokens into an AST.

Parsing is significant as it allows ambiguity as it defines context free grammar rules, retraining the safety component of this. Also, it solves the iteration gap as it is one of the gaps from elm-mustache.

2.4.1 Parser Generators: ML-Lex and ML-Yacc

: In the early 1990s, Appel, Tarditi and Mattson developed ML-Lex and ML-Yacc as standard ML parser-generator tools shipped with SML/NJ [48]. They follow the lex/yacc tradition originating in Unix [49], [50]: the developer writes a declarative grammar specification, and the tool generates SML source code that implements the corresponding state machine. ML-Yacc implements the LALR(1) algorithm, which is "Look Ahead Left to Right" parsing with one token of lookahead. This "bottom-up" algorithm reduces input to grammar productions as it scans left-to-right, and can handle a substantial subset of context-free grammars, including most programming language syntaxes [51]: providing a significant leverage as ML-Yacc is performed during generation time, meaning conflicts are reported as warnings before compilation. This forces errors to be solved in the grammar rather than after, catching a bug that hand-written parsers cannot.

2.4.2 Modern Application: ML-LPT ML-ULex and ML-Antlr

ML-LPT, the Language Processing Tools suite developed by Aaron Turon as part of the Manticore project, is the modern successor to ML-Lex and ML-Yacc [52]. It consists of

ML-ULex (a Unicode-aware lexer generator) and ML-Antlr (an LL(k) parser generator). ML-LPT was designed to address several limitations of the older tools: it produces purely functional code without imperative state, supports Unicode input properly, generates more readable output, and offers better error messages. ML-Antlr uses LL(k) parsing rather than LALR(1). LL(k) is a "top-down" algorithm: the parser predicts which grammar rule to expand based on the next k tokens, then recursively descends into that rule. This is closer to how a human reads a grammar and makes the generated parser easier to debug. However, LL grammars cannot directly express left-recursive rules (a production like $expr ::= expr" + "expr$, and must be refactored into right-recursive form or use iteration constructs. For some language designs this is awkward; for a logic-less template grammar with no operator precedence to encode, it makes little practical difference.

2.4.3 Parser Combinators

Originating in Haskell with Hutton and Meijer's monadic parsing work [53], parser combinators express the grammar directly in the host-language code rather than in a separate specification file and are higher-order functions that compose primitive parsers into larger ones. This makes them flexible because the developer can use the full power of the host language to define parsing logic and a separate file is not needed. Parser combinators also typically lack static conflict detection: ambiguities only manifest as unexpected parse failures at runtime. And while modern combinator libraries have largely solved the performance problems of naive backtracking, the algorithmic class is generally weaker than LALR(1) for context-free grammars [54].

2.4.4 CM-Lex and CM-Yacc

CM-Lex and CM-Yacc are another parser method, and this stands out because it can mitigate errors in the compilation order: specifically, this strict order requires compiling the AST first, followed by the generated tokens, the lexer, the parser logic, and finally the main application. If you go back and add more tokens or change something, it may cause issues [55]. To solve this, the SML/NJ compiler introduced the Compilation Manager (CM), designed by Matthias Blume [55].CM determines the exact dependency graph of a project and compiles only the files that have changed. The integration between the generation tools and the build system was tightened to improve type safety and error localisation, leading to the creation of CM-Lex and CM-Yacc [56]. These tools generate closed functors instead of disembodied code, working seamlessly within the CM environment. Particularly CM-Lex and CM-Yacc in th SML/NJ has leverages like:

- Declarative Specification: The grammar sits in its own file as a small number of productions, making it easy to read, audit, and extend.
- CM-Yacc reports grammar conflicts at generation time, catching ambiguities before

they become runtime failures.

- Alignment with Literature: The project's primary tutorial reference (Crary's CM-Lex and CM-Yacc User's Manual) uses exactly these tools, which made the learning curve manageable for a developer new to parser generators [56].

CM itself is a less significant part of the value here. It tracks dependencies between .cm-lex, .cmyacc, and .sml files and re-invokes the generators automatically when the grammar changes [55]. Without CM, every grammar change would require manually re-running the lexer and parser generators in the correct order

3. Requirements and Analysis

3.1 Problem Statement

The preprocessing burden is the problem this project addresses: Mustache's logic-less design forces the developer to compute and embed every contextual flag by hand, with no compile-time check that the work is complete.

The aim is to build a preprocessor in Standard ML that performs this work automatically. Given a raw JSON document and a Mustache template, the preprocessor produces an enriched JSON document that an existing Mustache renderer can consume without modification.

The preprocessor sits in front of any Mustache implementation and does not require any modification of the Mustache specification. The use of Standard ML lets the type system and pattern matching guarantee that the transformations are exhaustively defined. If a case is missing, the code does not compile.

3.2 Functional Requirements

The system needs to do the following:

1. Parse Mustache templates. The system must take a Mustache template string and turn it into an abstract syntax tree, supporting the standard tag types: variables, unescaped variables, sections, inverted sections, comments, partials, and plain text.

2. Read JSON input. The system must read JSON, either from a file or a string, and represent it internally using SML algebraic datatypes that match JSON's value space (objects, arrays, strings, numbers, booleans, null). I am using the SML/NJ JSON library for this, which I have already confirmed works on my machine.

3. Apply contextual enrichment. The system must transform the JSON data by adding contextual metadata that Mustache cannot compute on its own own. Two transformations are demonstrated in this project. First, every 'null' field in an object is rewritten so that the field carries an empty string value and a companion 'has' boolean flag is added alongside it; this lets templates use '{{#has_x}}' sections to handle optional fields. Second, every element of every JSON array is wrapped with 'isFirst' and 'isLast' boolean flags; this lets templates suppress trailing commas or mark the first item differently. Both transformations apply recursively, so deeply nested objects and arrays are enriched as well.

4. Produce usable output. The system must render a parsed Mustache AST against the en-

riched JSON, producing a final output string. The renderer must support variable lookup with dot-separated paths ('person.name'), the implicit iterator ('.'), HTML-escaping of escaped variables, sections, inverted sections, and comments.

5. Provide a command-line interface. The full pipeline must be runnable from the command line, taking a template file and a data file as arguments and writing the rendered output to standard output.

6. Preserve the original data. Enrichment must only add information, never delete or alter it. Reordering, type changes, or value modifications outside of the documented 'null, handling rule are not allowed.

3.3 Non-Functional Requirements

1. Compile-time correctness. The transformations must be written using pattern matching that the SML compiler can check for exhaustiveness. If I add a new case to the JSON datatype later and forget to handle it somewhere, the code should refuse to compile rather than silently break at runtime.

2. Determinism. For a given input, the output must be identical every time. There should be no reliance on hashing, map iteration order, or anything else non-deterministic.

3. Modularity. Each component (parser, preprocessor, renderer) must be independently testable and replaceable. Adding a new preprocessing transformation should not require changes to the parser or renderer, and vice versa.

3.4 Problem Decomposition

The system splits into five sub problems, as shown on Figure X. Each is implemented and tested independently, with the components composed together by a single command-line entry point.

3.4.1 Parsing the Mustache Template

A raw template string must be turned into a structured AST before anything else can happen. This breaks down into four pieces.

Lexing. The file "mustache.cmlex", compiled to "mustache.cmlex.sml" via ml-lex, scans the input character stream and emits tokens such as "SECTION", "CLOSE_SECTION", "INVERTED", "OPEN_TAG", "OPEN_UNESC", "PARTIAL", "COMMENT", "TEXT", "LBRACE", and "EOF". The lexer uses a lazy stream so tokens are generated on demand.

Parsing. The file "mustache.cmyacc", compiled to "mustache.cmyacc.sml" via ml-yacc, consumes the token stream using a context-free grammar with two nonterminals ("Template"

and "Node") and ten productions. Semantic actions such as "mk_text", "mk_section", and "mk_inverted" build the AST bottom-up. Arbitrary nesting of sections is supported.

AST. The shared datatype is defined in "ast.sml" with eight node constructors: "Text", "Variable", "UnescapedVariable", "Section", "InvertedSection", "Comment", "Partial", and "Template".

Orchestration. The file "mustache.sml" wires the lexer and parser together, exposes the public interface "Mustache.parse" (which takes a string and returns an "Ast.node"), and implements helpers such as "extractName" and "stripLeadingNewline".

3.4.2 Enriching the JSON Data

This is the core contribution of the project: pre-computing the logic Mustache cannot express, and embedding the results directly in the data. All enrichment lives in "preprocessing/preprocess.sml".

Null and optional fields ("processFields"). For every field "x" in a JSON object, if the value is null, the preprocessor replaces it with an empty string and adds a companion field "has_x = false". Otherwise, the value is kept and "has_x = true" is added. Templates can then write "{{#has_x}}...{{/has_x}}" to handle the optional case.

Array position flags ("flagList", "wrapItem"). For every element of every JSON array, the preprocessor adds "isFirst" and "isLast" boolean flags. Templates can then use "{{^isLast}}" to suppress trailing commas, or treat the first element specially.

Recursive traversal ("processValue"). Both transformations apply recursively across the whole JSON tree, so nested objects and arrays are enriched as well.

The public interface is "Preprocess.returnArray", which takes a "JSON.t" and returns a "JSON.t".

3.4.3 Rendering the Template

Once the AST and the enriched JSON are both available, the renderer combines them. All rendering logic lives in "renderer/renderer.sml".

Context resolution ("resolve", "lookupKey"). Looks up values in the JSON context. Supports dot-separated paths such as "person.name" and the implicit iterator ".".

Truthiness ("isTruth"). Decides whether a value triggers a "{{#section}}" block. "JSON.NULL", "JSON.BOOL false", and the empty array "JSON.ARRAY []" are falsy; everything else is truthy.

Node-by-node traversal ("renderNote"). Walks the AST recursively. "Text" nodes emit literal text; "Variable" nodes are looked up, converted to a string, and HTML-escaped; "Un-

escapedVariable" nodes are looked up without escaping; "Section" nodes either iterate over an array, render their body once for a truthy value, or skip for a falsy value; "InvertedSection" nodes render only when their value is falsy; "Comment" nodes are discarded, and "Template" nodes concatenate the rendered output of their children.

The public interface is "Render.render", which takes a string and a "JSON.t" and returns a string.

3.4.4 CLI and Build System

The components above are composed by a single command-line entry point. The file "renderer/main.sml" reads the template file and the data file, runs the preprocessing pipeline, calls "Render.render", and prints the result.

A build script ("renderer/build") generates a "render.cm" file with the correct absolute paths for "CMTOOL_HOME" and invokes "ml-build". A short shell wrapper ("renderer/render") launches the resulting heap image. The heap image suffix is currently platform-dependent (".amd64-darwin" on macOS, ".amd64-linux" on Linux), which is a known portability friction point discussed in Section 3.6.

3.4.5 Testing

Each component has its own test suite, written so that it can be run independently in the SML/NJ REPL. The parser suite ("mustache_parser/tests.sml") covers more than 21 cases including variables, sections, nesting, and newline-stripping behaviour. The preprocessor suite ("preprocessing/tests_preprocess.sml") covers 13 cases including null-field handling, position flags, and edge cases. The renderer suite ("renderer/tests_renderer.sml") covers 13 cases including HTML-escaping, sections, dotted lookup, and iteration.

During the implementation phase I considered adding comment handling into the parser, but decided this would introduce unpredictable edge cases that were not worth the additional complexity. I also chose not to support delimiter changes: although they are part of the Mustache specification, they are rarely used in practice, and supporting them was not necessary to demonstrate the project's core contribution.

3.5 Known Gaps and Limitations

A few aspects of the system are deliberately out of scope or remain partial. These are documented here so the evaluation in Chapter Z has a clear baseline.

Partials are parsed but not rendered. The '{{>file}}' syntax produces a 'Partial' node in the AST, but the renderer currently returns the empty string for these nodes.

Delimiter changes are parsed but not rendered. The '{{=« »=}}' syntax is recognised by the parser but ignored at render time.

Platform-specific heap image suffix. The shell wrapper hard-codes '.amd64-darwin' or '.amd64-linux'; using '$SMLNJ_ARCH' would resolve this but has not been implemented.

Hard-coded REPL paths. The development helper 'loadrender.sml' contains absolute paths under '/usr/local/smlnj/', which breaks on non-standard installs.

SML/NJ library not in Ubuntu apt repositories. On Linux, SML/NJ must be built from source to obtain '$/json-lib.cm'. This is a deployment friction rather than an implementation gap.

These limitations do not affect the core contribution of the project, the type-checked preprocessing pipeline but they are relevant to anyone trying to deploy the tool more widely, and they inform of further work discussion in Chapter Y.

3.6 Evaluation Strategy

I need to evaluate two things: whether the implementation behaves correctly, and whether the overall approach actually addresses the original aim of reducing the preprocessing burden.

Functional testing. Each component is tested in isolation against hand-written expected outputs, using SML's structural equality. The existing test suites described in Subproblem 5 satisfy this requirement.

Integration testing. The full pipeline is exercised end-to-end: a raw JSON document and a Mustache template are fed through the preprocessor and the renderer, and the final output is compared against an expected string. This confirms the enriched JSON is consumable by the rendering stage and that the components compose correctly through a couple different JSON test documents.

Comparison. To assess whether the project meets its original aim, I compare two ways of producing the same rendered output: once using my SML preprocessor, and once using JSON manually, prepared the way a developer would normally do it. The comparison looks at the lines of code required, the kinds of errors each approach allows, and whether the typed approach catches mistakes that the manual one misses. Based on the literature in Chapter 2, I expect the typed approach to catch missing or malformed flags at compile time rather than at render time.

What this project does not evaluate. I am not benchmarking the preprocessor against existing Mustache implementations on speed. The aims is correctness, not performance. I am also not running user studies on developer productivity, as that is out of scope for a single-author final year project, and the test JSON is beginner-made.

3.7 Legal and Ethical Considerations

This project does not involve any human participants, personal data, or confidential information. All the tools used (SML/NJ, CM-Lex, CM-Yacc, the SML/NJ JSON library) are open source under permissive licences that allow academic use and redistribution. The full source code is published openly on GitHub [57].

The work itself produces deterministic transformations of structured data and does not generate content of an ethically sensitive nature. No ethics approval is required from the School.

4. Design

This chapter explains the major design decisions behind the implementation described in Chapter 3, and justifies each choice against the alternatives that were considered. The chapter follows the order of the pipeline: language and architecture first, then parser, preprocessor, renderer, and finally build and deployment.

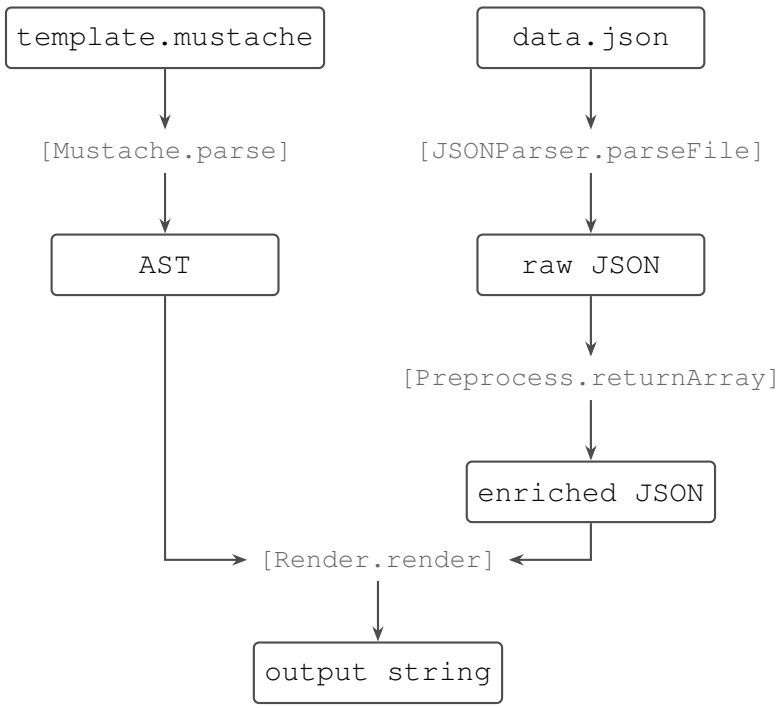

Figure 4.1: The preprocessing pipeline. The Mustache template is parsed into an abstract syntax tree (AST); the JSON data is parsed into a raw JSON value; the preprocessor walks the raw JSON and emits an enriched JSON value containing the contextual flags the template requires; the renderer consumes both the AST and the enriched JSON to produce the final output string.

4.1 Choice of Implementation Language

The first decision was which language to write the preprocessor in. The literature review identified Standard ML, OCaml, Haskell, and Elm as potential functional languages to utilise for this project, with dynamically-typed languages (JavaScript, Python, Ruby) as the alternative paradigm. I chose Standard ML. Why?

The first reason is the formal specification. Standard ML has a complete mathematical definition of its typing rules and operational semantics [46], and multiple standard-conforming implementations exist (SML/NJ, MLton, MLKit, Poly/ML) [58]. For a project whose central claim is about type-checked correctness, working in a language with a formal specification

means that the guarantees the type system provides are precisely defined, not just behaviour observed in one implementation.

The second reason is pattern-match exhaustiveness. Standard ML's algebraic datatypes are a natural fit for representing both Mustache's AST and JSON's value space, but the same is broadly true of Haskell, OCaml, and Elm. What Standard ML adds is a compiler that refuses to compile a function if its pattern match misses any constructor of the datatype. If I add a new case to the JSON datatype later and forget to handle it somewhere in the preprocessor, the code does not compile. Appel describes this as one of the core advantages of statically-typed functional languages "adding a new constructor to a datatype... will cause compile-time errors everywhere that the pattern matching is incomplete" [47]. For a project whose contribution is about catching errors at compile time rather than through testing, this guarantee is essential.

The third reason is simplicity. Standard ML's design is small and coherent. It evaluates strictly, so the time and space behaviour of a program is straightforward to reason about, unlike Haskell's lazy evaluation. It does not bolt on an object system, unlike OCaml. This simplicity matters because a dissertation prototype is not the place to be fighting language complexity.

A disadvantage of choosing Standard ML is that its ecosystem offers far fewer third-party libraries than Python or JavaScript, and noticeably fewer than OCaml or Haskell. This project depends on the SML/NJ JSON library, which proved sufficient for the preprocessing task but has a much smaller user base than equivalents such as Python's'simplejson' or Haskell's 'aeson'. As a result, fewer edge cases have been encountered and resolved by the wider community, and the available documentation is comparatively sparse

A dynamically-typed language was rejected for the obvious reason: the project's contribution is precisely the use of a static type system to catch preprocessing errors at compile time. A Python preprocessor could do all the same data transformations, but it would not provide the exhaustiveness guarantees, which is the whole point.

Elm, which already has a Mustache implementation [45], was the most serious alternative. Elm pairs a strict type system with logic-less templates and demonstrates that the architectural pairing is viable. However, Elm is primarily a frontend language compiled to JavaScript, and its Mustache library does not automate preprocessing transformations: developers still write the data construction by hand. Standard ML, potentially used as a backend preprocessing tool, lets the project focus on automating the transformation rather than on browser integration.

4.2 Architectural Choice: Preprocessor Rather Than New Engine

The second major decision was where to put the tool in the rendering pipeline. There were three options:

1. Modify an existing Mustache implementation to add the missing logic.
2. Build a new template engine that supports the missing operations directly.
3. Build a preprocessor that sits in front of any unmodified Mustache implementation.

I chose the third. Each of the alternatives had a problem that ruled it out for this project.

Modifying an existing Mustache implementation would tie the work to a specific language ecosystem (e.g. JavaScript fork), and would fragment the Mustache specification. Mustache is interesting precisely because the same template can be rendered by implementations in dozens of languages. This would not work with its syntax of being 'logic-less', alternatively would mean just manual labour.

Building a new template engine would make it possible to support conditionals, arithmetic, and other operations directly in the template, but doing so would re-introduce exactly the security risks identified by Parr (2004) [13] and Pisu et al. (2024) [5]. The motivation for working with Mustache is its logic-less design; an engine that adds logic would not preserve that property and would defeat the purpose. Additionally Mustache's verstality into many languages was one of its strong suits, it made more sense to utlise this for the long-run.

The JSON data would be operated on before any template engine sees it, the engine itself remains unchanged, retraining its advantages. The output is just JSON, with the contextual flags embedded as ordinary fields, so it works with the JavaScript reference implementation, the Ruby implementation, the Haskell implementation, or any other. The Mustache specification is preserved.

The trade-off is that the preprocessor cannot help with logic that Mustache itself rejects. Anything that needs to be visible to the template must be pre-computed and embedded. In practice this is exactly the burden the project aims to automate, so the trade-off is aligned with the goal.

4.3 Parser Design

The parser turns a raw template string into an 'Ast.node' value. Three sub-decisions matter: how to lex, how to parse, and how to shape the AST.

4.3.1 Parser Generators

I chose to use CM-Lex and CM-Yacc, the parser-generator tools that ship with SML/NJ, rather than writing the lexer and parser by hand or using parser combinators. Mustache parser must handle nested sections, balanced opening and closing tags, distinct tag types (variables, unescaped variables, sections, inverted sections, comments, partials), and cases like newline-stripping around standalone tags.

Parser combinators (the more idiomatic functional-programming alternative) were rejected for a different reason: they make the grammar implicit in the code. With CM-Yacc, the grammar sits in its own file as ten productions over two nonterminals, which makes the parser's behaviour easy to read, easy to extend, and easy for a future reader to verify against the Mustache specification. This also follows the approach used in published SML language-processing work, including Crary's CM-Lex/CM-Yacc manual which served as the primary reference.

4.3.2 AST Shape

The AST is defined in 'ast.sml' as a single algebraic datatype with nine constructors:

```
structure Ast =
struct
    datatype node
        = Text of string
        | Variable of string
        | UnescapedVariable of string
        | Section of string * node list
        | InvertedSection of string * node list
        | Comment of string
        | Partial of string
        | DelimiterChange of string * string
        | Template of node list
end
```

This shape was chosen because it maps directly onto Mustache's surface syntax. Each constructor corresponds to one tag type in the specification, which makes the parser's job mostly mechanical: each grammar production emits exactly one node constructor. The only constructors that hold recursive structure are 'Section', 'InvertedSection', and 'Template', which is the minimum needed to express arbitrary nesting.

Two of the constructors ('Partial' and 'DelimiterChange') are present because the parser recognises their syntax, but the renderer does not process them. I chose to keep them in the AST rather than reject them at parse time so that the parser remains a faithful representation

of Mustache syntax, with rendering decisions made later. This means the AST can be reused by future work that does implement partials or delimiter changes, without re-parsing.

4.4 Preprocessor Design

The preprocessor is the core contribution of the project. Its design has three parts: the choice of representation, the choice of transformations, and the shape of the recursive traversal.

4.4.1 Operating on the JSON Datatype

The preprocessor operates on the SML/NJ JSON library's 'JSON.t' datatype rather than on JSON as a raw string. This is important: pattern matching on the datatype is what makes the type-system guarantees work. A string-based preprocessor could perform the same transformations using regular expressions, but the compiler would have no way to verify that every JSON value type was handled. By operating on the typed datatype, the compiler verifies exhaustiveness over 'OBJECT', 'ARRAY', 'STRING', 'NUMBE', 'BOOL', and 'NULL'. If the JSON library adds a new constructor in a future version, the preprocessor will fail to compile until that case is handled. This is precisely the property the project is designed to demonstrate.

4.4.2 Public Interface

The preprocessor exposes a small set of functions:

```
val wrapItem      : JSON.t -> bool -> bool -> JSON.t
val flagList      : JSON.t list -> JSON.t list
val processFields : (string * JSON.t) list -> (string * JSON.t) list
val processValue  : JSON.t -> JSON.t
val returnArray   : JSON.t -> JSON.t
```

'wrapItem' attaches 'isFirst' and 'isLast' flags to a single value. 'flagList' applies it across a list, computing the correct flags for each position. 'processFields' handles object fields, applying the null-rewriting rule. 'processValue' is the recursive entry point. 'returnArray' is the public-facing function that consumers call.

'processFields' and 'processValue' are mutually recursive, which is what allows the transformation to descend through arbitrarily nested JSON in a single pass. Each function handles one level of structure (object fields, or any value), and they call each other to walk into nested contents.

4.4.3 A Worked Example

The following example illustrates all the transformation rules acting together. The input contains a present field, a null field, and an array of strings:

```
{
  "name": "Snow Leopard",
  "description": null,
  "threats": ["habitat loss", "poaching"]
}
```

After 'Preprocess.returnArray', the same data has been enriched with contextual metadata:

```
{
  "name":            "Snow Leopard",
  "has_name":        true,
  "description":     "",
  "has_description": false,
  "threats": [
    { "value": "habitat loss", "isFirst": true,  "isLast": false },
    { "value": "poaching",     "isFirst": false, "isLast": true  }
  ],
  "has_threats": true
}
```

A Mustache template can now use '{{#has_description}}' to render an optional block, or '{{^isLast}}, {{/isLast}}' to suppress the trailing comma on the last array element. None of this logic exists in the template, and none of it had to be computed by the application code.

4.5 Renderer Design

The renderer takes a parsed AST and an enriched JSON value and produces the final output string. Three design decisions are worth justifying: the choice of a tree-walking interpreter, the definition of truthiness, and the support for dot-separated paths.

4.5.1 Tree-Walking Interpreter

The renderer is implemented as a tree-walking interpreter rather than as a compiler that produces an intermediate form (a sequence of closures, say, or a bytecode). This is the simplest approach. The AST is small, walking it directly is straightforward, and the performance cost, which would matter for a production template engine doing millions of renders per second, is irrelevant for a dissertation prototype focused on correctness.

The renderer is structured around a single recursive function that dispatches on each node

type. Each constructor has exactly one branch, and the SML compiler verifies that no branch is missing. This direct mapping from AST shape to renderer behaviour makes the renderer's correctness easy to argue.

4.5.2 Truthiness

Mustache's specification leaves some latitude in what counts as a "falsy" value. I chose the following definition, taken verbatim from 'renderer.sml':

```
fun isTruth JSON.NULL           = false
  | isTruth (JSON.BOOL false)   = false
  | isTruth (JSON.ARRAY [])     = false
  | isTruth _                   = true
```

A value is falsy if it is JSON null, the boolean 'false', or the empty array; everything else is truthy. This matches the behaviour of most mainstream Mustache implementations.

The empty-array case is the most subtle of the three. Without it, '{{#items}}...{{/items}}' would render its body once for an empty array, which is almost never what a template author wants. Including the empty-array case in the falsy set means that '{{^items}}No items.{{/items}}' works correctly as a fallback message.

4.5.3 Dot-Separated Path Resolution

The renderer supports dot-separated paths such as '{{person.name}}' in addition to single-segment lookups. Real JSON data is typically nested, and forcing template authors to flatten their data structure before rendering would re-introduce a form of preprocessing burden, which is the opposite of what the project is trying to do. Supporting dotted paths costs only a handful of lines in 'resolve' and 'lookupKey' and broadens the range of templates the system can render usefully.

4.6 Build and Deployment

The final design decision was how to package the tool: as a library callable from other SML code, or as a command-line tool. I chose the command-line tool because the most common use case for a preprocessor of this kind is shell-based scripting:

```
./render template.mustache data.json > output.html
```

This is the same workflow used by other Mustache command-line tools and by classic Unix text processors, so it fits naturally into existing toolchains. Wrapping the pipeline in a single binary also means users do not need to know any Standard ML to use the tool.

The build process uses 'ml-build' to produce a heap image, which is then launched by a

short shell wrapper. The heap image's filename suffix is platform-specific ('.amd64-darwin' on macOS, '.amd64-linux' on Linux), which the wrapper currently hard-codes. This is a known limitation, documented in Section 3.5, and would be straightforward to fix using the '$SMLNJ_ARCH' environment variable in future work. I did not fix it during the project because it is a deployment friction rather than a correctness issue, and the time was better spent elsewhere.

5. Implementation and Testing

This chapter describes how the design from Chapter 4 was actually built, the implementation challenges that took meaningful time to resolve, and the testing carried out at each stage. The full source code is available in GitHub [57].

5.1 Implementation Overview

The project is implemented in Standard ML and uses three external tools: SML/NJ (Standard Meta Language/ New Jersey) as the compiler, CM-Lex and CM-Yacc as the lexer and parser generators, and Git for version control. The repository is organised into three top-level directories matching the three subsystems described in Chapter 3: 'mustache_parser/' for the lexer, parser, AST, and orchestration code; 'preprocessing/' for the JSON enrichment; and 'renderer/' for context resolution, truthiness, and node-by-node rendering. Each directory contains its own test suite, runnable independently from the SML/NJ REPL.

Development was carried out on macOS using SML/NJ version 110.99.9. One quirk of the local environment is that the system's default 'sml' command invokes Poly/ML rather than SML/NJ, so SML/NJ has to be invoked through its absolute path. This matters because the SML/NJ JSON library used by the preprocessor is not available under Poly/ML. The build script handles this internally, but it is documented here because anyone reproducing the build will hit the same issue.

5.2 Implementing the Parser

The parser was the first component built. Every other component depends on the AST it produces, and my supervisor had recommended using CM-Lex and CM-Yacc as a learning vehicle for parser generators before doing anything else.

The lexer, defined in 'mustache.cmlex', scans the input character stream and emits tokens for each Mustache construct. It is implemented as a lazy stream, which means tokens are only produced when the parser asks for them. This was conceptually unfamiliar at first, because lazy evaluation in SML requires a specific construction: a thunk of type 'unit -> 'a front' wrapped inside a 'Stream' constructor. The 'Stream' wrapper is not a design choice but a syntactic requirement of SML. Mutual recursion needs the wrapper to type-check correctly. Once the pattern was understood it became straightforward to extend.

The parser, defined in 'mustache.cmyacc', consumes the token stream through a context-free grammar with two nonterminals ('Template' and 'Node') and ten productions. Semantic ac-

tions in the grammar invoke constructor functions ('mk_text', 'mk_section', 'mk_inverted', 'mk_variable', and so on) that build AST nodes bottom-up as the parser reduces. One non-obvious detail of CM-Yacc is that grammar precedence numbers behave the opposite way to the BIDMAS (order of brackets, indices, division, multiplication, addition, subtraction) convention I was used to: higher numbers bind more tightly, not less. This caused some early confusion when writing the grammar but is documented in Karl Crary's CM-Lex/CM-Yacc manual.

' wires the lexer and parser together and exposes the public interface 'Mustache.parse'. It also implements two helpers: 'extractName', which strips whitespace and the leading sigil from a tag's contents, and 'stripLeadingNewline', which removes the leading newline from the body of a section so that templates do not produce blank lines around section boundaries.

5.3 Implementing the Preprocessor

The preprocessor lives in 'preprocessing/preprocess.sml' and is the core contribution of the project. It consists of three main functions: 'processFields', 'flagList' (helped by 'wrapItem'), and 'processValue'.

'processFields' walks the fields of a JSON object. For each field, if the value is 'JSON.NULL', the value is replaced with an empty string and a companion field of the form 'has_x = false' is added (where 'x' is the original field name). If the value is anything else, it is kept and the companion field is set to 'has_x = true'. This makes optional fields expressible in a Mustache template as '{{#has_name}}...{{/has_name}}'.

'flagList' walks a JSON array and rewraps every element so that it carries 'isFirst' and 'isLast' boolean flags. The wrapping is done by 'wrapItem', which takes the original value and the two flags and produces a new 'JSON.OBJECT' containing them. The implementation tracks position by pattern-matching on the head/tail structure of the list: the first element gets 'isFirst = true', the last element gets 'isLast = true', and everything else gets both flags set to false.

'processValue' ties the two transformations together by recursively walking the entire JSON tree. Whenever it encounters an 'OBJECT', it applies 'processFields' and then recurses into the object's values. When it encounters an 'ARRAY', it applies 'flagList' and then recurses into each element. This recursive traversal is the reason deeply nested structures get enriched correctly, and it is also the part of the code where the exhaustiveness check earns its keep, every branch over the 'JSON.t' datatype must be present, which makes it impossible to forget a case.

The public interface is 'Preprocess.returnArray', which takes a 'JSON.t' and returns a 'JSON.t' with all transformations applied. Exposing only this single function keeps the rest of the

codebase decoupled from the internal helpers.

5.4 Implementing the Renderer

The renderer, in 'renderer/renderer.sml', takes a parsed AST and an enriched JSON value and produces the rendered output string. It consists of four pieces: a context resolver, a truthiness function, an HTML escape function, and the main 'renderNode' walker.

'resolve' and 'lookupKey' together implement context lookup. They support dot-separated paths (so 'person.name' walks into the 'person' object and then reads its 'name' field) and the implicit iterator '.' (which refers to the current item when iterating over an array).

'isTruth' decides whether a value triggers a section block. The rule is given in Section 4.5.2.

'htmlEscape' replaces the five HTML-significant characters (&, <, >, ", ') with their corresponding entity references. It is called only on the output of 'Variable' nodes, not 'UnescapedVariable' nodes, which is why Mustache distinguishes the two with double versus triple braces.

'renderNode' is the main walker. It takes a node and a context and produces a string. 'Text nodes return their stored text directly; 'Variable' nodes look up the name and HTML-escape it; 'UnescapedVariable' nodes do the same without escaping; 'Section nodes either iterate over an array (rendering the body once per element with the element as the new context), render the body once for a truthy non-array value, or skip entirely for a falsy value; 'InvertedSection nodes render only when the value is falsy; 'Comment nodes return the empty string; and 'Template' nodes concatenate the output of their children.

The public interface is 'Render.render, which takes a template string and a 'JSON.t' and returns a string. Internally it parses the template, runs the preprocessor on the JSON, and walks the resulting AST.

5.5 Coding Traps Encountered

Several issues took time to resolve and are documented here so anyone building on this work does not lose the same hours.

CMtool installation. The CMtool source distribution did not include the 'Makefile' and 'calc.cm' files referenced in the documentation, and the generated lexer and parser file names ('calc.cmlex.sml', 'calc.cmyacc.sml') differ from the names suggested in older tutorials ('calc.lex.sml', 'calc.parse.sml'). Resolving this required reading the generator output rather than trusting the documentation, and writing the build configuration files by hand.

Unbound signatures with the standard library. On a clean install, '$/smlnj-lib.cm' did not resolve correctly when used inside 'calc.cm', producing an unbound signature error. The

fix was to hard-code the absolute path to 'cmlib.cm' inside the project directory. This is a portability issue rather than a logic bug, but it consumed non-trivial time.

Default 'sml invokes the wrong implementation. As already noted, the default 'sml' command on this development machine invokes Poly/ML, not SML/NJ. Since the JSON library is specific to SML/NJ, the build and REPL workflows always require the absolute path '/usr/local/smlnj/bin/sml'. A 'loadrender.sml' helper file was written to automate REPL setup, but its paths are hard-coded and do not work on non-standard installs.

Platform-specific heap image suffix. The shell wrapper that launches the renderer hard-codes '.amd64-darwin' as the heap image suffix. On Linux, this would need to be '.amd64-linux'. A more portable solution would substitute '$SMLNJ_ARCH' at runtime, but this has not been implemented.

SML/NJ JSON library not in apt. On Ubuntu the SML/NJ packages in apt do not include the JSON library, so SML/NJ has to be built from source to obtain '$/json-lib.cm'.

None of these issues affected the correctness of the implementation, but together they form most of the deployment story for the tool and explain several entries in the further-work discussion in Chapter 7.

5.6 Testing Strategy

Each subsystem has its own test 'assert', runnable independently from the SML/NJ REPL. Keeping the suites separate (rather than maintaining a single combined one) reflects the modular structure of the codebase: the parser can be tested without the renderer, the preprocessor can be tested without either, and so on. This made debugging significantly easier during development, because a failure in one suite localises the problem immediately.

The test pattern in all three suites is the same. Each test calls the function under test with a known input, handwrites the expected output, and uses SML's structural equality to check whether the actual result matches. Failures are reported with the test name. This pattern was adopted in preference to a full testing framework because Standard ML's structural equality already provides the comparison primitive, and adding a framework would have introduced a dependency for no real benefit at this project's scale.

There is also different ranges of test JSONs: short and simple, large class scenario, large animal scenario.

5.7 Test Results

All three test suites pass at the time of writing.

The **parser suite** ('mustache_parser/tests.sml') covers 21 cases including simple variables, unescaped variables, sections, inverted sections, nesting at multiple depths, and the newline-stripping rules.

The **preprocessor suite** ('preprocessing/tests_preprocess.sml') covers 13 cases including null-field handling (both single and multiple fields in the same object), array position flags (single element, two elements, many elements, empty array), and recursive traversal into nested structures.

The **renderer suite** ('renderer/tests_renderer.sml') covers 13 cases, including HTML-escaping of the five significant characters, section iteration over arrays, dotted-path lookup, the implicit iterator, inverted sections, and edge cases such as missing keys and empty contexts.

Integration testing was carried out by passing several JSON documents and Mustache templates through the command-line interface end-to-end and comparing the rendered output against expected strings. These tests confirmed that the components compose correctly and that enriched JSON produced by the preprocessor is consumable by the renderer without modification.

It is worth being explicit about the limits of this testing. The JSON inputs used were written by me to exercise the cases of the preprocessor was designed for: present and null fields, arrays of various lengths, and a few levels of nesting. They do not stress-test the system against unusual or pathological JSON shapes, arbitrarily deep recursion, very large arrays, unusual Unicode in keys, and so on. Thus the code works strictly for the tests I have run it on. A more extensive test corpus would be needed before the prototype could be considered production-ready, and this is picked up explicitly in the further-work discussion.

5.8 Limitations Confirmed by Testing

Two known gaps surfaced through testing. Partials and delimiter changes are both parsed correctly into the AST but ignored by the renderer. The parser tests confirm that '{{>file}}' produces a 'Partial' node and that '{{=« »=}}' is recognised as a delimiter change, but the renderer tests confirm that both currently render as empty strings. These gaps were intentional, as discussed in Chapter 3 and revisited in Chapter 7, but they are real limits on the prototype's coverage of the Mustache specification.

6. Results and Discussion

This chapter presents the outcomes of the project and evaluates the approach against the original aims, compares the typed preprocessing pipeline against manual preprocessing, and identifies the directions for further work that the project has surfaced. The presentation is deliberately honest about scope: the tool is a prototype, not a production system, and the results should be read in that light.

6.1 Summary of Findings

The project delivers a working preprocessing tool that sits in front of any standard Mustache implementation and automates the data preparation that Mustache's logic-less design would otherwise require developers to do by hand. The full pipeline runs end-to-end from a raw JSON file and a Mustache template through to the rendered output, invokable from a single command-line entry point.

The implementation comprises three subsystems, each with its own test suite. The parser handles all eight Mustache node types (variables, unescaped variables, sections, inverted sections, comments, partials, and plain text, organised under a top-level 'Template' node). The preprocessor implements two transformations: null-field handling, which rewrites null values and adds companion 'has_x' flags, and array position flagging, which adds 'isFirst' and 'isLast' flags to every element in every JSON array. Both transformations apply recursively across nested structures. The renderer walks the AST against the enriched JSON and produces the final output string.

All 47 unit tests across the three subsystems pass, and end-to-end integration tests confirm that the preprocessor's output is consumable by the renderer without modification. The tool runs successfully on macOS; the Linux build path has been documented but not tested exhaustively.

6.2 Goals Achieved

The project's original aim was to investigate whether Standard ML's type system could automate the preprocessing transformations required by logic-less templates while preserving the security and separation guarantees of logic-less design. The implementation demonstrates that the answer is yes, with qualifications worth being clear about.

Type-checked exhaustiveness. The preprocessor's transformations are expressed as pattern matches over the JSON datatype. The SML compiler verifies that every constructor of

'JSON.t' is handled by every relevant function. In practice this caught omissions during development that would have manifested as runtime errors in a dynamically-typed language, and it is the property the literature review identified as the central appeal of typed functional languages for this task.

Mustache compatibility preserved. Because the preprocessor operates on JSON in, JSON out, the rendered output is consumable by any standard Mustache implementation, not only the renderer built in this project. This was a non-negotiable design constraint and it was achieved.

Recursive generalisation. The two transformations apply recursively through nested objects and arrays without requiring the developer to think about traversal. This is a meaningful improvement over the manual approach, where developers must remember to handle nested cases in their own preparation code, and where forgetting to do so produces silent rendering failures.

Test coverage within scope. The 47 passing tests provide reasonable confidence that the implementation behaves correctly across the JSON shapes the preprocessor was designed for. They do not provide confidence that the implementation handles inputs outside that range, which is the most significant honest limit on the results.

Goals partially achieved. Two parts of the Mustache specification were intentionally not implemented. Partials are parsed into the AST but rendered as empty strings, because supporting them would have required an orthogonal file-loading mechanism that does not relate to the type-safety contribution. Delimiter changes are similarly recognised by the parser but ignored by the renderer, on the grounds that they are rarely used in practice and would have expanded scope without strengthening the core argument.

6.3 Comparative Analysis

To assess whether the typed preprocessing approach reduces developer effort compared to manual preprocessing, the same template was rendered twice: once using the SML preprocessor, and once using JSON manually prepared the way a developer using vanilla Mustache would have to prepare it.

For a representative input containing one nullable field and one nested array of five items, the manual approach required twelve manual augmentations: three 'has_x' flags (one per nullable field), and nine flags across the array ('isFirst' and 'isLast' on each of the five elements, plus one for the array itself). With the SML preprocessor, the developer wrote zero. The preprocessor produced identical output to the manual approach but with two important differences.

First, the manual approach offered no protection against omission. A developer who forgot

to add an 'isLast' flag to one of the five array items would produce a render with a stray comma at the end of a list, and would only discover the bug by inspecting the rendered output. The typed approach makes this class of error structurally impossible: the recursive walk over the JSON datatype is exhaustive by construction, and the SML compiler enforces this exhaustiveness.

Second, the manual approach is fragile under change. Adding a new field to the JSON structure requires the developer to remember to add a 'has_x' flag for it, in every place that consumes the data. The typed approach requires no change at all: the new field will be picked up automatically by 'processFields' because the function is defined recursively over the datatype rather than over a hardcoded list of field names.

These are qualitative findings rather than quantitative ones. The project did not run user studies, and the comparison above is a single representative example rather than a measured benchmark. What it does establish is that the typed approach eliminates a category of error rather than merely reducing its likelihood.

There is also a comparative note worth making about code size. The SML preprocessor is around 50 lines of code that handles arbitrary nesting and arbitrary numbers of fields and array elements. The manual equivalent for a non-trivial JSON document, particularly one that changes shape between renders, can run to several hundred lines of bespoke flag-setting code, which has to be maintained as the data shape changes. The point is not that 50 lines is impressive in itself; it is that the preprocessor's size is a constant rather than a function of the data, which is the property that makes it useful.

6.4 Discussion

A few observations are worth recording even though they were not framed as formal aims at the start of the project.

The implementation effort was unevenly distributed. The parser produced the most code but was the most mechanical to write, because CM-Lex and CM-Yacc do most of the heavy lifting once the grammar is correct. The preprocessor, which is the core contribution, took proportionally less code than expected: the recursive structure of pattern matching over algebraic datatypes makes the transformations remarkably compact. Most of the implementation time was spent on the renderer, on the build and deployment plumbing, and on the environment quirks documented in Chapter 5.

The choice of Standard ML proved appropriate but not effortless. The compile-time guarantees worked as the literature predicted: pattern exhaustiveness caught errors of omission early and consistently. The cost was the ecosystem friction, the absence of SML/NJ from Ubuntu's apt repositories, the platform-specific heap image suffix, the Poly/ML default on

macOS. These costs are real and would matter for anyone considering deploying this approach in production, but they did not threaten the validity of the contribution.

A wider observation is that the preprocessing burden identified in Chapter 2 is not a single problem but a family of related ones. The two transformations implemented here are the most commonly cited examples in the literature, but other patterns exist: alternating-row markers, count totals, reverse-position flags, sorted-order indicators. The architecture supports these naturally. Each is a new function over 'JSON.t' composed into 'processValue', but actually implementing them was out of scope for this dissertation. This generalisation is the single most promising direction for further work.

Finally, an honest reflection on testing. The 47 passing tests are a real result, but they exercise the cases I designed the preprocessor to handle. They do not exercise inputs I did not anticipate, and they do not approximate the diversity of real-world JSON. Anyone reading the test results should treat them as evidence of correctness within scope, not as evidence of robustness.

6.5 Further Work

Several directions for future work have emerged from the project.

Extending the transformation library. The two transformations implemented here are a small fraction of the patterns developers use when preparing data for logic-less templates. A natural extension would be to build a wider library, alternating-row flags, total counts, sort-order indicators, conditional aggregation flags, and to investigate whether these can be composed into reusable pipelines. The type system should make composition mechanically straightforward, but the question of which transformations to support is empirical rather than formal.

Implementing partials. Partials were left unrendered because file loading is orthogonal to the type-safety contribution. A future version could implement them as a separate loading pass that runs before preprocessing, expanding '{{>file}}' references into their target template's AST and continuing as before. This would not affect the type-checked guarantees on the data side.

Implementing delimiter changes. Delimiter changes are rarely used but are part of the specification. Adding support would require threading a delimiter pair through the lexer's state, which is mechanically straightforward.

Improving deployment portability. The platform-specific heap image suffix, the hard-coded paths in 'loadrender.sml', and the requirement that SML/NJ be built from source on Ubuntu, all make the tool harder to deploy than necessary. A '$SMLNJ_ARCH'-aware build script and a Windows port would significantly lower the barrier to use.

Wider testing. The most honest gap in the current evaluation is the limited diversity of the JSON test inputs. A future study could build a corpus of real-world JSON documents drawn from existing Mustache-using projects, run the preprocessor over them, and identify failure modes that the hand-written test suite did not exercise. This is the work that would turn the prototype into something with credible robustness claims.

Quantitative evaluation. The comparative analysis here is qualitative. A future study could quantify the reduction in developer effort across a larger sample of templates and JSON structures, and could attempt to measure the rate at which manual preprocessing produces silent rendering errors in real codebases. This would require either a user study or a corpus analysis, both of which were out of scope.

Generalisation beyond Mustache. The same approach could in principle be applied to other logic-less template engines, most obviously Liquid (the Shopify engine discussed in Chapter 2). The preprocessor itself is template-engine-agnostic. It transforms JSON so the extension would primarily involve adapting the renderer.

7. Conclusions

This dissertation set out to investigate whether Standard ML's type system could automate the preprocessing burden imposed by logic-less template systems, with Mustache as the working example. The preceding chapters have described the literature that motivated the question, the requirements derived from that literature, the design decisions behind a prototype implementation, the implementation itself, and the results of testing it.

The core contribution is a working prototype: a Mustache parser, a JSON preprocessor with two type-checked transformations, and a renderer, composed by a single command-line entry point. The preprocessor sits in front of any standard Mustache implementation, preserves the cross-language portability that makes Mustache useful, and uses pattern-matching exhaustiveness to ensure that the transformations are applied consistently across nested data structures. All 47 unit tests pass within the scope they were designed to cover, and end-to-end integration tests confirm the components compose correctly.

The contribution is not a production-ready tool. It is evidence that the typed-functional approach to preprocessing automation is viable for the cases it was designed for, and that the architecture generalises naturally to additional transformations. Two parts of the Mustache specification (partials and delimiter changes) are parsed but not rendered. The deployment story has rough edges that would matter at scale: the build script hard-codes a platform-specific heap image suffix, several development paths are absolute rather than relative, and the SML/NJ JSON library has to be built from source on Ubuntu. The test corpus is limited in variety and exercises the cases the preprocessor was designed for; it does not stress-test the system against unusual JSON shapes that a real-world deployment would encounter.

These limitations are real, but they do not undermine the contribution. The point of a prototype is to demonstrate that an approach is worth pursuing, and the prototype shows three things. First, that a typed functional language can express the transformations that logic-less templates require, in code whose size is a constant of the language rather than a function of the data. Second, that pattern-matching exhaustiveness catches errors of omission at compile time rather than at render time, which is the property the literature identified as missing from existing systems. Third, that the preprocessor architecture decouples cleanly from any particular Mustache implementation, which means the approach is in principle deployable in any of the dozens of language ecosystems that already use Mustache.

The most significant directions for further work are widening the transformation library beyond the two patterns implemented here, testing the system against a corpus of real-world JSON to identify failure modes the hand-written test suite did not exercise, and quantifying

the reduction in developer effort that the typed approach provides. Each of these would turn evidence-of-viability into evidence-of-robustness, which is the next step a follow-on project should take.

The wider implication, if the approach generalises, is that the choice between security and usability that Parr (2004) described as fundamental to template-system design is not actually as binary as the literature has presented it. Mustache makes the secure choice and pays in preprocessing work; logic-friendly engines make the usable choice and pay in security vulnerabilities. The prototype here suggests that the cost of the secure choice can be substantially reduced by moving the preprocessing into a typed functional language where the compiler does the work of catching omissions. Whether that suggestion holds up at production scale is a question for further work, but the prototype shows it is worth asking.

A. Appendix Title

Appendices may be provided to include further details of results, mathematical derivations, certain illustrative parts of program code (e.g. class interfaces), user documentation, or a log of project milestones. In particular, if there are technical details of the work done that might be useful to others who wish to build on this work, but that are not sufficiently important to the project as a whole to justify being discussed in the main body, then they should be included here.

Do **not** include an appendix containing all your source code listings , this material will be collected electronically. If there are any code fragments of particular novelty, you may include these so they can be referenced from the main text; this is analogous to including appendices for mathematical proofs.

Note: Appendices do not count towards the page limit, but equally they are not treated as part of the dissertation for assessment purposes. There is no expectation that examiners will read the appendices. Any material significant to judging the quality of the dissertation must be in the main body, not in appendices.

Should your project require ethics approval from the School, it is **compulsory** to include in your dissertation: a copy of the ethics application, all supporting documents, and the official ethics approval.

Bibliography

[1] Helm Contributors, *Helm: The package manager for Kubernetes*, Official Website. [Online]. Available: https://helm.sh/.

[2] F. Y. Rashid, *Infrastructure as code templates are the source of many cloud infrastructure weaknesses*, CSO Online, Jan. 2024. [Online]. Available: https://www.csoonline.com/article/568909/infrastructure-as-code-templates-are-the-source-of-many-cloud-infrastructure-weaknesses.html.

[3] Pallets, *Jinja*, Official Website, 2008. [Online]. Available: https://palletsprojects.com/p/jinja/.

[4] Wikipedia contributors, *FreeMarker*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: https://en.wikipedia.org/wiki/FreeMarker.

[5] L. Pisu, D. Maiorca, and G. Giacinto, "An assessment of the overlooked dangers of template engines," *arXiv preprint arXiv:2405.01118*, Mar. 2025. DOI: https://doi.org/10.1145/3799796. arXiv: 2405.01118 [cs.CR]. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3799796.

[6] PortSwigger, *Server-side template injection*, Web Security Academy, Accessed: 12 May 2026, 2015. [Online]. Available: https://portswigger.net/web-security/server-side-template-injection.

[7] OWASP Foundation, *OWASP Top 10: 2021*, Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/Top10/.

[8] Evalian, *OWASP top 10 security vulnerabilities 2021*, Evalian Blog, Oct. 2021. [Online]. Available: https://evalian.co.uk/owasp-top-10-security-vulnerabilities-2021/.

[9] J. Kettle, *Server-side template injection*, PortSwigger Research, Whitepaper accompanying Black Hat USA 2015 presentation, Aug. 2015. [Online]. Available: https://portswigger.net/research/server-side-template-injection.

[10] O. Tsai, *uber.com may RCE by Flask Jinja2 Template Injection*, HackerOne Report #125980, Mar. 2016. [Online]. Available: https://hackerone.com/reports/125980.

[11] A. Tiurin, *Exploiting SSTI in Thymeleaf*, Acunetix Web Security Blog, Jun. 2020. [Online]. Available: https://www.acunetix.com/blog/web-security-zone/exploiting-ssti-in-thymeleaf/.

[12] Y. Zhao, Y. Zhang, and M. Yang, "Remote code execution from SSTI in the sandbox: Automatically detecting and exploiting template escape bugs," in *Proc. 32nd USENIX*

Security Symposium (USENIX Security 23), USENIX Association, 2023, pp. 3691–3708.

[13] T. J. Parr, "Enforcing strict model-view separation in template engines," in *Proceedings of the 13th International Conference on World Wide Web*, ser. WWW '04, New York, NY, USA: ACM, May 2004, pp. 224–233, ISBN: 1-58113-844-X. DOI: 10.1145/988672.988703.

[14] *Mustache manual*, Mustache, Sep. 2022. [Online]. Available: https://mustache.github.io/mustache.5.html.

[15] Mustache Organization, *Mustache github repository*, GitHub, 2009. [Online]. Available: https://github.com/mustache/mustache.

[16] Mustache, *Mustache - logic-less templates*, Official Website, 2009. [Online]. Available: https://mustache.github.io/.

[17] S. Pottavathini, *The case against logic-less templates*, eBay Inc. Engineering Blog, Oct. 2012. [Online]. Available: https://innovation.ebayinc.com/stories/the-case-against-logic-less-templates/.

[18] A. Boronine, *Cult of logic-less templates*, Alexei Boronine's Blog, Sep. 2012. Accessed: Sep. 7, 2012. [Online]. Available: https://www.boronine.com/2012/09/07/Cult-Of-Logic-less-Templates/.

[19] S. Pullara, *Mustache is logic-less but the logic has to go somewhere*, Medium, Jun. 2016. [Online]. Available: https://medium.com/spullara/mustache-is-logic-less-but-the-logic-has-to-go-somewhere-3e976b7a49c8.

[20] M. Samuel, P. Saxena, and D. Song, "Context-sensitive auto-sanitization in web templating languages using type qualifiers," in *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS '11)*, Chicago, Illinois, USA: ACM, Oct. 2011, pp. 587–600. DOI: 10.1145/2046707.2046775.

[21] A. Agarwal, S. Mondet, P. Veber, C. Troestler, and F. Berenger, "Biocaml: The OCaml bioinformatics library," in *OCaml Users and Developers Meeting (OUD)*, Copenhagen, Denmark, Sep. 2012. [Online]. Available: http://biocaml.org/.

[22] F. Berenger, K. Y. J. Zhang, and Y. Yamanishi, "Chemoinformatics and structural bioinformatics in OCaml," *Journal of Cheminformatics*, vol. 11, no. 1, p. 24, Feb. 2019. DOI: 10.1186/s13321-019-0332-0. [Online]. Available: https://jcheminf.biomedcentral.com/articles/10.1186/s13321-019-0332-0.

[23] Wikipedia contributors, *Smalltalk Wikipedia, the free encyclopedia*, https://en.wikipedia.org/wiki/Smalltalk, [Online; accessed 14-May-2026], 2026.

[24] G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80," *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, Aug. 1988.

[25] K. Gao, Y. Sun, and C.-A. Sun, *Understanding bugs in template engine-based applications: Symptoms, root causes, and fix patterns*, 2018. arXiv: 2604.27692 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2604.27692.

[26] Wikipedia contributors, *Mustache (template system)*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Mustache_(template_system).

[27] J-Gallo. "Mustache vs Handlebars. "[Online]. Available: https://medium.com/@JuaniGallo/mustache-vs-handlebars-42e531eca252.

[28] Shopify, *Liquid: Safe, customer-facing template language for flexible web apps*, https://shopify.github.io/liquid/, 2024.

[29] F. Potencier, *Twig: The flexible, fast, and secure template engine for php*, https://twig.symfony.com, 2024.

[30] Mozilla, *Nunjucks: A powerful templating engine with inheritance*, GitHub, 2012. [Online]. Available: https://mozilla.github.io/nunjucks/.

[31] GitHub Advisory Database, *CVE-2019-10906: Jinja2 sandbox escape via string formatting*, https://github.com/advisories/GHSA-462w-v97r-4m45, Accessed: [Insert Date], 2019.

[32] National Vulnerability Database, *CVE-2024-56326: Jinja sandboxed environment indirect call bypass*, https://nvd.nist.gov/vuln/detail/CVE-2024-56326, Accessed: [Insert Date], 2024.

[33] D. Thomas, C. Fowler, and A. Hunt, *Programming Ruby: The Pragmatic Programmers' Guide*, 4th. Raleigh, NC: Pragmatic Bookshelf, 2013.

[34] EJS Team, *Ejs – embedded javascript templates*, Official Website, 2010. [Online]. Available: https://ejs.co/.

[35] React Team, *Writing markup with jsx*, React Documentation, 2023. [Online]. Available: https://react.dev/learn/writing-markup-with-jsx.

[36] Svelte Team, *Svelte documentation*, Official Website, 2016. [Online]. Available: https://svelte.dev/docs.

[37] Scalate Project, *Scalate: Scala template engine*, https://scalate.github.io/scalate/, Includes Mustache support for the Play Framework, 2024.

[38] D. VandenBerghe, *The case for ml*, comp.compilers forum, republished on Yale FLINT website, Jul. 1998. [Online]. Available: https://flint.cs.yale.edu/cs421/case-for-ml.html.

[39] Wikipedia contributors, *ML (programming language)*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: https://en.wikipedia.org/wiki/ML_(programming_language).

[40] M. Snoyman, *Hamlet: Type-safe templates via Template Haskell*, Yesod Web Framework, 2012. [Online]. Available: https://www.yesodweb.com/book/shakespearean-templates.

[41] J. Van Der Jeugt, *Blazehtml: A blazingly fast html combinator library*, Personal Website, 2010. [Online]. Available: https://jaspervdj.be/blaze/.

[42] S. Peyton Jones, *Haskell 98 Language and Libraries: The Revised Report*. Cambridge: Cambridge University Press, 2003.

[43] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, and J. Vouillon, *The ocaml system release 4.02: Documentation and user's manual*, Institut National de Recherche en Informatique et en Automatique, 2014.

[44] T. Bunko, *Jingoo: Template engine for ocaml*, GitHub, 2013. [Online]. Available: https://github.com/tategakibunko/jingoo.

[45] L. Wendel, *Elm-mustache: A mustache implementation for elm*, https://package.elm-lang.org/packages/lukewestby/elm-string-interpolate/latest/, Logic-less template rendering in a strictly typed functional language, 2018.

[46] R. Milner, M. Tofte, R. Harper, and D. MacQueen, *The Definition of Standard ML (Revised)*. Cambridge, MA: MIT Press, 1997.

[47] A. W. Appel, *Modern Compiler Implementation in ML*. Cambridge, UK: Cambridge University Press, 1998.

[48] A. W. Appel, J. S. Mattson, and D. R. Tarditi, "A lexical analyzer generator for Standard ML," Department of Computer Science, Princeton University, Tech. Rep., 1994, Documentation for ML-Lex, distributed with SML/NJ.

[49] S. C. Johnson, "Yacc: Yet another compiler-compiler," Bell Laboratories, Murray Hill, NJ, Tech. Rep. Computing Science Technical Report No. 32, 1975.

[50] M. E. Lesk and E. Schmidt, "Lex: A lexical analyzer generator," Bell Laboratories, Murray Hill, NJ, Tech. Rep. Computing Science Technical Report No. 39, 1975.

[51] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd. Boston, MA: Pearson Education, 2006.

[52] A. Turon, *SML/NJ language processing tools: User guide*, Standard ML of New Jersey, 2009. [Online]. Available: https://www.smlnj.org/doc/ml-lpt/manual.pdf.

[53] G. Hutton and E. Meijer, "Monadic parser combinators," *Journal of Functional Programming*, vol. 8, no. 4, pp. 437–444, 1996.

[54] Wikipedia contributors, *Parser combinator*, Wikipedia, The Free Encyclopedia, [Online; accessed 15-May-2026], 2026. [Online]. Available: https://en.wikipedia.org/wiki/Parser_combinator.

[55] M. Blume, "The sml/nj compilation manager (cm) user manual," Lucent Technologies, Bell Labs, Tech. Rep., 2000. [Online]. Available: https://www.smlnj.org/doc/CM/new.pdf.

[56] K. Crary, *Cm-lex and cm-yacc user's manual (version 2.1)*, Carnegie Mellon University, Dec. 2017. [Online]. Available: https://www.cs.cmu.edu/~crary/cmtool/manual.pdf.

[57] J. Y. Lew, *Templating in Standard ML: Project repository*, GitHub, 2025. [Online]. Available: `https://github.com/jylew1/Templating-in-SML`.

[58] SML Family, *Standard ml family GitHub project*, GitHub Pages, 2015. [Online]. Available: `https://smlfamily.github.io/`.